

Backpropagation

Gradient Descent (batch)

1. Inicialize a Rede Neural com pesos aleatórios
2. Para cada instancia dos dados x_i (i-ésima linha) da matriz X
 - (a) Faça o Fordward Pass, ou seja, calcule a ativação de cada neurônio da rede, usando para as hidden layers a função de ativação sigmoide e para a output layer a sua respectiva função de ativação que dependera do problema, ou seja, calcule o valor final que cada neurônio possuirá (por exemplo para a camada L com n neurônios e a camada (L-1) com m neurônios devemos ter $z_{i1}^{(L)}, z_{i2}^{(L)}, \dots, z_{in}^{(L)}$ onde:
 $z_{ij}^{(L)} = \sigma(a_{ij}^{(L)})$, para $j = 1, 2, \dots, n$
 $a_{ij}^{(L)} = w_{j0}^{(L-1)} z_{i0}^{(L-1)} + w_{j1}^{(L-1)} z_{i1}^{(L-1)} + \dots + w_{jm}^{(L-1)} z_{im}^{(L-1)}$ e
 $w_{j0}^{(L-1)} z_{i0}^{(L-1)} = w_{j0}^{(L-1)} \cdot 1$ é o bias, ou seja aqui tratamos o primeiro neurônio de cada camada como o bias, então seu valor ativado já é 1, isso é conhecido como bias trick)

- (b) Calcule δ para todos os Neurônios na Output Layer, usando a predição (ou seja a saída final dada pela output layer) e o target.

Problema de regressão (saída escalar ou vetorial, SSE, identidade):

$$\delta_{ij}^{(Out)} = z_{ij}^{(Out)} - t_{ij}, \quad j = 1, \dots, \dim(Out)$$

Problema de classificação binaria (Cross Entropy, sigmoid):

$$\delta_{ij}^{(Out)} = z_{ij}^{(Out)} - t_{ij}, \quad j = 1, \dots, \dim(Out)$$

Problema de classificação multiclasse, clases não mutuamente exclusivas (Binary Cross Entropy, sigmoide):

$$\delta_{ij}^{(Out)} = z_{ij}^{(Out)} - t_{ij}, \quad j = 1, \dots, \dim(Out)$$

Problema de classificação multiclasse, clases mutuamente exclusivas (Categorical Cross Entropy, Softmax):

$$\delta_{ij}^{(Out)} = z_{ij}^{(Out)} - t_{ij}, \quad j = 1, \dots, \dim(Out)$$

Percebeu? calculando a derivada do erro para qualquer um desses problemas o calculo do delta da saída é o mesmo. Por quê? não sei o porquê.

Então para uma instancia x_i dos dados cada neurônio da Output layer tera um valor δ associado a ele, este δ , para apenas um neurônio, para uma única instancia dos dados x_i é um numero real (então perceba, quando eu tenho uma entrada de dados x_i , um neurônio especifico tem 3 coisas: seu valor desativado a seu valor ativado z e sua taxa de variação instantânea do erro total em relação a seu valor desativado a $\frac{\partial E}{\partial a} = \delta$)

- (c) Calcule δ para todos os Neurônios nas camadas anteriores:

$$\delta_{ij}^{(L)} = \sum_k \delta_{ik}^{(L+1)} w_{kj}^{(L+1)} z_{ij}^{(L)} (1 - z_{ij}^{(L)}), \quad j = 1, \dots, \dim(L) \quad k = 1, \dots, \dim(L+1)$$

- (d) Calcule o Gradiente da Função de Erro em relação a todos os pesos $w_{jk}^{(L)}$, $k \neq 0$ e $w_{j0}^{(L)}$ (bias da camada L). (Pode ser utilizada apenas a formula da esquerda tomando $a_{i0}^{L-1} = 1$)

$$\frac{\partial E_{x_i}}{\partial w_{jk}^{(L)}} = \delta_{ij}^{(L)} z_{ik}^{(L-1)}, \quad \frac{\partial E_{x_i}}{\partial w_{j0}^{(L)}} = \delta_{ij}^{(L)} \quad j = 1, \dots, \dim(L) \quad k = 1, \dots, \dim(L-1)$$

3. Calcule a media do Gradiente em relação a cada Peso e a cada Bias sobre todo o Conjunto de Treinamento (Pode ser utilizada a formula unificada):

$$\frac{\partial E}{\partial w_{jk}^{(L)}} = \frac{1}{n} \sum \frac{\partial E_{x_i}}{\partial w_{jk}^{(L)}} \quad \frac{\partial E}{\partial w_{j0}^{(L)}} = \frac{1}{n} \sum \frac{\partial E_{x_i}}{\partial w_{j0}^{(L)}}$$

4. Atualize os Pesos e os Bias usando Gradient Descent (novamente, isso pode ser feito tudo junto, apenas com a formula da esquerda):

$$w_{jk}^{(L)} \rightarrow w_{jk}^{(L)} = w_{jk}^{(L)} - \eta \frac{\partial E}{\partial w_{jk}^{(L)}} \quad w_{j0}^{(L)} \rightarrow w_{j0}^{(L)} = w_{j0}^{(L)} - \eta \frac{\partial E}{\partial w_{j0}^{(L)}}$$

5. Repita os passos 2-4 ate que o Erro reduza a um nível aceitável, cada uma destas repetições dos passos 2 ate 4 é chamada de Epoch, ou seja, uma Epoch é fazer os passos 2, 3 e 4

Stochastic Gradient Descent

1. Inicialize a Rede Neural com pesos aleatórios
2. Para cada instancia dos dados x_i (i-ésima linha) da matriz X
 - (a) Faça o Fordward Pass, ou seja, calcule a ativação de cada neurônio da rede, ou seja, calcule o valor final que cada neurônio possuirá (por exemplo para a camada L devemos ter $a_1^{(L)}, a_2^{(L)}, \dots, a_n^{(L)}$ onde:
$$a_i^{(L)} = \sigma(z_i^{(L)}),$$

$$z_i^{(L)} = w_{i0}^{(L-1)} a_0^{(L-1)} + w_{i1}^{(L-1)} a_1^{(L-1)} + \dots + w_{im}^{(L-1)} a_m^{(L-1)} \text{ e}$$

$$a_0^{(L)} = 1 \text{ (é o bias)}$$
 - (b) Calcule δ para todos os Neurônios na Output Layer, usando a predição (ou seja a saída final dada pela output layer) e o target. Ambos (predição e target), no caso de uma única saída para cada instancia, serão um vetor ou, uma matriz caso haja múltiplas saídas para cada instancia:

$$\delta_j^{(L)} = 2 \left(\hat{a}_j^{(L)} - a_j^{(L)} \right) a_j^{(L)} (1 - a_j^{(L)})$$

- (c) Calcule δ para todos os Neurônios nas camadas anteriores. Dada nossa matriz $W^{(L+1)}$ (é a que transforma $A^{(L)}$ em $Z^{(L+1)}$ por meio da multiplicação matricial $Z^{(L+1)} = A^{(L)} W^{(L+1)}$) cada linha representa um neurônio da camada $L+1$ (pois usamos a notação TO-FROM) e cada coluna representa um neurônio da camada L , ou seja a j-ésima coluna da matriz $W^{(L+1)}$ são todos os pesos que o j-ésimo neurônio da camada L ira multiplicar. A soma a seguir começa a partir de $j = 1$ pois w_{k0}^{L+1} representa o bias da camada L (ou seja ignoramos)

$$\delta_j^{(L)} = \sum_k \delta_k^{(L+1)} w_{kj}^{(L+1)} a_j^{(L)} (1 - a_j^{(L)})$$

- (d) Calcule o Gradiente da Função de Erro em relação a todos os pesos w_{jk}^L , $k \neq 0$ e w_{j0}^L (bias da camada L). (Pode ser utilizada apenas a formula da esquerda tomando $a_0^{L-1} = 1$)

$$\frac{\partial E_{x_i}}{\partial w_{jk}^{(L)}} = \delta_j^{(L)} a_k^{(L-1)} \quad \frac{\partial E_{x_i}}{\partial w_{j0}^{(L)}} = \delta_j^{(L)}$$

- (e) Atualize os Pesos e os Bias usando Stochastic Gradient Descent (novamente, isso pode ser feito tudo junto, apenas com a formula da esquerda):

$$w_{jk}^{(L)} \rightarrow w_{jk}^{(L)} = w_{jk}^{(L)} - \eta \frac{\partial E_{x_i}}{\partial w_{jk}^{(L)}} \quad w_{j0}^{(L)} \rightarrow w_{j0}^{(L)} = w_{j0}^{(L)} - \eta \frac{\partial E_{x_i}}{\partial w_{j0}^{(L)}}$$

3. Repita os passos 2 ate que o Erro reduza a um nível aceitável, cada uma das repetições do passo 2 é chamada de Epoch, ou seja, uma Epoch é fazer o passo 2 inteiro